# Audspect BAS — Signing Infrastructure

**Classification:** Internal — Audspect Engineering / Confidential
**Platform Version:** v1.7.5

## Overview

Audspect BAS uses **two completely separate signing systems** with two separate keys. Do not conflate them:

| | Content Signing | Bundle Signing |
|---|---|---|
| **Protects** | Scenario YAML files, `BINARIES.sha256` agent manifest | The delivery ZIP itself (transport integrity) |
| **Mechanism** | Native Go `crypto/rsa` (RSA-4096 + SHA-256, PKCS1v15) — **not GPG** | Real GPG (RSA-4096) |
| **Signing key** | `orchestrator/private_key.pem` | GPG keyring, `releases@audspect.com` |
| **Signing tool** | `orchestrator/scripts/signer.go` | `gpg` (inline in `windows-build.ps1` step 9b, or `packaging/signing/sign.sh` for a Linux-side alternative) |
| **Verified by** | The orchestrator binary itself, at startup, every boot | The customer, manually, before unzipping (`verify-sig.sh`) |

| | | |
|---|---|---|
| **Public key location** | Compiled into the binary (`internal/integrity/signing.go`'s `ScenarioPublicKeyPEM` const) | `pubkey.asc`, staged alongside the ZIP |
| **Expiry** | None — a raw RSA keypair, no PGP expiry concept | 2029-06-04 |

---

# Content Signing (Scenarios + Manifest)

## How It Works

`orchestrator/scripts/signer.go` is a small standalone Go program (`go run`, not a compiled release artifact) with two subcommands:

```
go run orchestrator/scripts/signer.go keygen
go run orchestrator/scripts/signer.go sign private_key.pe
```

`keygen` calls `rsa.GenerateKey(rand.Reader, 4096)` directly — there's no passphrase, no PGP wrapper, and critically **no expiry**. `sign` computes a SHA-256 hash of the file and produces a base64-encoded raw `rsa.SignPKCS1v15` signature (not an ASCII-armored PGP block) written to `<file>.sig`.

`windows-build.ps1`'s step 0b runs `keygen` once (idempotent — skipped if `orchestrator/private_key.pem` already exists) and then re-signs every scenario YAML on every build, so edits are always captured. `BINARIES.sha256` is signed separately in step 5c, after it's extracted from the built Docker image.

## Verification (Runtime, Automatic)

The **public** key is a plain PKIX PEM block, hardcoded as the `ScenarioPublicKeyPEM` constant in `orchestrator/internal/integrity/signing.go` — compiled into the binary, never read from disk at runtime. At startup, `VerifyScenarioFile` (called from `LoadScenarios()`) and `LoadManifestVerified` each independently re-derive the SHA-256 hash of the file and call `rsa.VerifyPKCS1v15` against it. There is no `gpg --verify` subprocess call anywhere in this path — this is pure native Go crypto, self-contained in the binary.

- Missing `.sig` file → scenario marked unsigned, cannot be dispatched
- Signature doesn't verify → scenario marked "signature invalid," cannot be dispatched
- `BINARIES.sha256` signature invalid → binary trust verification is disabled with a startup warning (not a hard failure)
- If `ScenarioPublicKeyPEM` is still the placeholder `"SIGNING_KEYGEN_REQUIRED"` (a pre-release/dev build that never ran `keygen`), verification is skipped entirely

A 15-second filesystem watcher (`internal/integrity/watcher.go`) re-checks loaded scenario files on that interval and alerts in the dashboard if a previously-valid scenario's content changes on disk after load — catching in-place tampering, not just load-time forgery.

## Backing Up `private_key.pem`

**Losing `orchestrator/private_key.pem` means every previously-shipped scenario and manifest can never be validly re-signed under that lineage** — the next release would need a new keypair, a new embedded public key, and a full re-sign of everything, shipped as a breaking change to every existing install (see Emergency section below). Back it up to an encrypted offline medium or a secrets vault; never commit it to git.

## Rotating the Content Signing Key

1. Delete (or move aside) `orchestrator/private_key.pem` and

re-run `windows-build.ps1` step 0b — a fresh keypair is generated
2. Copy the new public key PEM into `ScenarioPublicKeyPEM` in `internal/integrity/signing.go`
3. The next full build re-signs every scenario and the manifest with the new key automatically
4. Ship the new orchestrator version — until customers upgrade, their existing (old-key-signed) scenarios continue to verify fine against their still-old binary; the break only happens if an old binary somehow receives new-key-signed content, which the normal upgrade path never does

---

# Bundle Signing (Delivery ZIP)

## Current Key

| Attribute | Value |
|---|---|
| Algorithm | RSA-4096 (GPG) |
| Fingerprint | `D8B7F9581EAD62026F57A17E7B6F1A541AF7E0F5` |
| UID | `Audspect BAS Platform (BAS Release Signing Key) <releases@audspect.com>` |
| Created | 2026-06-05 |
| Expiry | **2029-06-04** — renew or replace before this date; start the process 3 months out |
| Location | Windows build host GPG keyring |
| Backup | **CRITICAL — must be backed up off the build host** |

## How It Works

`windows-build.ps1` step 9b signs the finished `bas-install-`

`<version>.zip` with this key (gracefully skipped, not a build failure, if GPG or the key isn't available), producing `bas-install-<version>.zip.asc`. It also stages a verify kit (`pubkey.asc` + `verify-sig.sh`) in `dist\` so the customer can authenticate the ZIP themselves, before unzipping — this is the only signature check a human ever runs manually; everything else in this document is automatic.

`packaging/signing/keygen.sh`, `sign.sh`, `verify-sig.sh`, `verify-binary.sh`, and `cosign.sh` are the Linux-side equivalents of this same GPG bundle-signing workflow (plus optional `cosign` container-image signing) — useful if signing needs to happen from a non-Windows host, but not what `windows-build.ps1` itself calls; its GPG signing is inline PowerShell.

## Backing Up the Key

```
gpg --export-secret-keys --armor D8B7F9581EAD62026F57A17E
gpg --export --armor D8B7F9581EAD62026F57A17E7B6F1A541AF7
```

Store the private export in an encrypted offline medium or a secrets vault (HSM/KMS-backed). **Never commit it to git. Never email it.**

## Key Rotation Procedure

**Trigger:** Expiry approaching (within 3 months of 2029-06-04) or suspected compromise.

1. Generate a new key: `gpg --full-generate-key` (RSA and RSA, 4096 bits, 3-year expiry)
2. Back it up per above
3. Update `pubkey.asc` wherever it's staged/published for customers
4. Future builds sign with the new key automatically (`windows-build.ps1` picks up whatever key matches `releases@audspect.com` in the keyring)
5. Announce the rotation in release notes so customers know to

expect a new `pubkey.asc`
6. Revoke the old key once the new release is out: `gpg --gen-revoke <OLD-KEY-ID> > old-key-revocation.asc`

## Emergency: Bundle Key Compromise

If the GPG bundle-signing key is believed compromised, this affects **transport integrity of future deliveries only** — it does not affect already-verified installs, and it is entirely separate from the content-signing key above (a compromised bundle key cannot forge a valid scenario or manifest signature).

1. Generate and back up a new key immediately
2. Re-sign and re-publish the current release's ZIP with the new key
3. Revoke the compromised key
4. Notify customers to re-verify their most recent download against the new `pubkey.asc`

---

*© Audspect Engineering — Internal / Confidential*